

07.1: Orchestration with Airflow

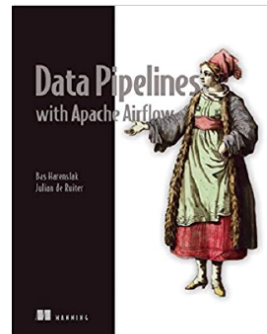


UMD DATA605 - Big Data Systems

07.1: Orchestration with Airflow

Instructor: Dr. GP Saggese, gsaggese@umd.edu

- Concepts in the slides
- Airflow tutorial
- Web resources
- Documentation
- Tutorial
- Mastery
- Data Pipelines with Apache Airflow

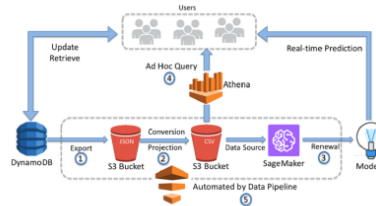


1 / 13

2 / 13: Workflow Managers

Workflow Managers

- **Data pipelines** move/transform data across stores
- **Orchestration problem** = coordinate jobs across systems
 - Run tasks on schedule
 - Run tasks in order (dependencies)
 - Monitor tasks
 - Notify devops on failure
 - Retry on failure
 - Track runtime
 - Meet real-time constraints
 - Scale performance



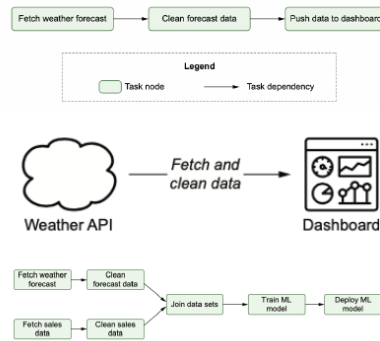
2 / 13

- **Data pipelines** are essential for moving and transforming data across different storage systems. They ensure that data flows smoothly from one point to another, often involving various transformations to make the data usable for different applications.
- The **orchestration problem** involves coordinating these data movements and transformations across multiple systems. This includes:
 - Running tasks on a specific schedule to ensure timely data processing.
 - Executing tasks in a specific order, respecting dependencies between them.
 - Monitoring tasks to ensure they are completed successfully. This involves:
 - * Notifying the devops team if a task fails.
 - * Retrying tasks automatically if they encounter errors.
 - * Tracking the runtime of tasks to optimize performance.
 - Meeting real-time constraints to ensure data is processed and available as needed.
 - Scaling performance to handle increasing data loads efficiently.
- The diagram illustrates a workflow where data is exported from DynamoDB to an S3 bucket in JSON format, converted to CSV, and then used by SageMaker for model training. Athena is used for ad hoc queries, and the entire process is automated by a data pipeline.

3 / 13: Workflow Managers

Workflow Managers

- **E.g., live weather dashboard**
 - Fetch weather data from API
 - Clean/transform data
 - Push data to dashboard/website
- Problems
 - Schedule tasks
 - Manage task dependencies
 - Monitor functionality and performance
 - Add machine learning quickly
 - Complexity increases quickly



3 / 13

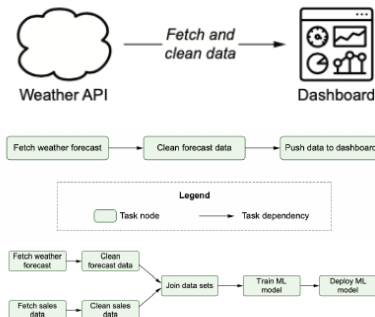
- **Workflow Managers**
 - Workflow managers are tools that help automate and manage complex processes. They are essential for tasks like maintaining a live weather dashboard.
- **Live Weather Dashboard Example**
 - *Fetch weather data from API:* This involves retrieving real-time weather information from an external source.
 - *Clean/transform data:* The raw data needs to be processed to ensure accuracy and usability.
 - *Push data to dashboard/website:* The cleaned data is then displayed on a user-friendly interface for end-users.
- **Problems**
 - *Schedule tasks:* Ensuring tasks run at the right time is crucial for real-time data updates.
 - *Manage task dependencies:* Tasks often rely on the completion of previous steps, requiring careful coordination.
 - *Monitor functionality and performance:* Continuous monitoring is needed to ensure the system runs smoothly.
 - *Add machine learning quickly:* Integrating machine learning models can enhance functionality but adds complexity.
 - *Complexity increases quickly:* As more features are added, managing the workflow becomes more challenging.

These points highlight the importance of workflow managers in handling complex data processes efficiently, ensuring timely and accurate data delivery.

4 / 13: Workflow Managers

Workflow Managers

- **Workflow managers address orchestration problem**
 - E.g., airflow, Luigi, Metaflow, make, cron
- **Represent data pipelines as DAGs**
 - Nodes are tasks
 - Direct edges are dependencies
 - Execute task when all ancestors executed
 - Execute independent tasks in parallel
 - Re-run failed tasks incrementally
- **Describe data pipelines**
 - Static files (e.g., XML, YAML)
 - Workflows-as-code (e.g., Python in Airflow)
- **Provide scheduling**



4 / 13

- **Workflow managers address orchestration problem**
 - Workflow managers like Airflow, Luigi, Metaflow, Make, and Cron help organize and manage complex processes. They ensure that tasks are executed in the correct order and handle dependencies between tasks.
- **Represent data pipelines as DAGs**
 - Data pipelines are visualized as Directed Acyclic Graphs (DAGs). In these graphs, nodes represent tasks, and directed edges show dependencies. A task runs only after all its prerequisite tasks are completed. Independent tasks can run simultaneously, and failed tasks can be retried without rerunning the entire pipeline.
- **Describe data pipelines**
 - Pipelines can be defined using static files like XML or YAML, or through code, such as Python scripts in Airflow. This flexibility allows for easy configuration and modification of workflows.
- **Provide scheduling**
 - Workflow managers allow users to specify what tasks to run and when. This scheduling capability is crucial for automating repetitive tasks and ensuring timely execution.
- **Provide backfilling and catch-up**
 - These tools can handle backfilling, which means running missed tasks, and catching up with delayed tasks. They are horizontally scalable, meaning they can distribute tasks across multiple runners to improve efficiency.
- **Provide monitoring web interface**
 - A web interface is often available for monitoring the status of tasks, viewing logs, and managing workflows. This interface provides a user-friendly way to oversee and troubleshoot workflows.

5 / 13: Airflow

Airflow

- Developed at AirBnB in 2015
 - Open-sourced as part of Apache project
- **Batch oriented framework** for building data pipelines (not streaming)
- **Data pipelines**
 - Represented as DAGs
 - Described as Python code
- **Scheduler with rich semantics**
- Web-interface for monitoring
- Large ecosystem
 - Support many DBs
 - Many actions (e.g., emails, pager notifications)
- **Hosted and managed solution**
 - Run Airflow on your laptop (e.g., in tutorial)
 - Managed solution (e.g., AWS)



5 / 13

- **Developed at AirBnB in 2015**
 - Airflow was created by AirBnB to manage complex workflows and data processing pipelines. It was later open-sourced and became part of the Apache Software Foundation, which helps in its development and maintenance.
- **Batch oriented framework for building data pipelines (not streaming)**
 - Airflow is designed to handle batch processing, meaning it processes data in large chunks at scheduled intervals, rather than continuously like streaming frameworks.
- **Data pipelines**
 - These are sequences of data processing steps. In Airflow, they are represented as Directed Acyclic Graphs (DAGs), which ensure that tasks are executed in a specific order without cycles.
 - Pipelines are described using Python code, making them highly customizable and easy to integrate with other Python-based tools.
- **Scheduler with rich semantics**
 - Airflow's scheduler is powerful and flexible, allowing for complex scheduling scenarios, such as running tasks at specific times or intervals.
- **Web-interface for monitoring**
 - Airflow provides a user-friendly web interface to monitor, manage, and troubleshoot workflows, making it easier to oversee the execution of data pipelines.
- **Large ecosystem**
 - Airflow supports integration with many databases and can perform various actions like sending emails or pager notifications, making it versatile for different use cases.
- **Hosted and managed solution**
 - Airflow can be run locally on a laptop for development and testing purposes. For produc-

tion, managed solutions are available, such as on AWS, which handle the infrastructure and scaling needs.

6 / 13: Airflow: Execution Semantics

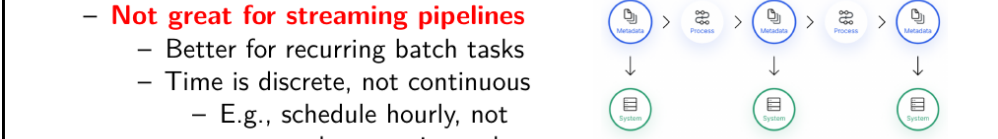
Airflow: Execution Semantics

- **Scheduling semantic**
 - Define next scheduling interval
 - E.g., “every day at midnight”, “every 5 minutes on the hour”
 - Similar to **cron**
- **Retry**
 - Re-run task after failure to recover from intermittent issues
- **Incremental processing**
 - Divide time into intervals per schedule
 - Execute DAG for data in that interval only
- **Catch-up**
 - Run all missing intervals up to now (e.g., after downtime)
- **Backfilling**
 - Execute DAG for past schedule intervals
 - E.g., re-process data after pipeline changes

6 / 13

- **Scheduling semantic**
 - This refers to how tasks are scheduled to run at specific times. You can set up tasks to run at regular intervals, like “every day at midnight” or “every 5 minutes on the hour.” This is similar to how *cron jobs* work, which are used in Unix-like systems to schedule commands or scripts to run periodically.
- **Retry**
 - If a task fails, Airflow can automatically try to run it again. This is useful for handling temporary issues that might cause a task to fail, such as network problems or temporary unavailability of a service. Retrying helps ensure that tasks eventually complete successfully without manual intervention.
- **Incremental processing**
 - This involves breaking down data processing into smaller, manageable chunks based on time intervals. For example, if you have a daily schedule, the Directed Acyclic Graph (DAG) will process only the data for each day separately. This approach helps in efficiently managing and processing large datasets over time.
- **Catch-up**
 - If your system goes down or tasks are missed, Airflow can catch up by running all the tasks that were supposed to run during the downtime. This ensures that no data processing is skipped and that the system remains up-to-date.
- **Backfilling**
 - This is the process of running tasks for past time intervals. It is useful when you need

- **Not great for streaming pipelines**
 - Better for recurring batch tasks
 - Time is discrete, not continuous
 - E.g., schedule hourly, not process data continuously
 - **Prefer static pipelines**
 - DAGs should remain consistent between runs
 - **No data lineage**
 - No automatic tracking of data transformation
 - Implement manually
 - **No data versioning**
 - No automatic tracking of data updates
 - Implement manually
- 
- 7 / 13



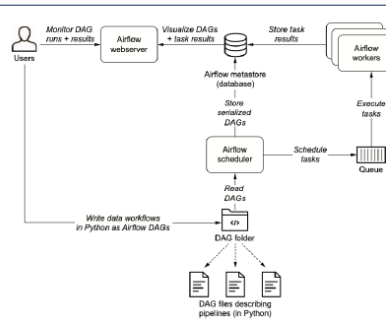
7/13

- **Not great for streaming pipelines**
 - Airflow is more suited for *recurring batch tasks* rather than continuous data processing. This means it works best when tasks are scheduled at specific intervals, like hourly or daily, rather than processing data in real-time. The concept of time in Airflow is *discrete*, meaning tasks are triggered at set times rather than continuously.
- **Prefer static pipelines**
 - Airflow is designed to work with static pipelines, where Directed Acyclic Graphs (DAGs) remain consistent between runs. This consistency is crucial for ensuring that the workflow behaves predictably each time it is executed.
- **No data lineage**
 - Airflow does not automatically track how data is transformed throughout the pipeline. This means users need to manually implement data lineage tracking if they need to understand how data changes from start to finish.
- **No data versioning**
 - Similar to data lineage, Airflow does not automatically track changes or updates to data over time. Users must manually implement data versioning to keep track of different data states or versions.

8 / 13: Airflow: Components

Airflow: Components

- **Users (DevOps)**
- **Web-server**
 - Visualize DAGs
 - Monitor DAG runs and results
- **Metastore**
 - Keep system state
 - Track executed DAG nodes
- **Scheduler**
 - Parse DAGs
 - Track completed dependencies
 - Add tasks to execution queue
 - Schedule tasks
- **Queue**
 - Tasks ready for execution
 - Tasks picked up by Workers
- **Workers**
 - Pick up tasks from Queue
 - Execute tasks
 - Register task outcome in Metastore



8 / 13

- **Users (DevOps)**
 - DevOps teams interact with Airflow to manage and monitor workflows.
 - They use the web interface to visualize and track the progress of data workflows.
- **Web-server**
 - *Visualize DAGs*: Provides a graphical interface to view Directed Acyclic Graphs (DAGs), which represent workflows.
 - *Monitor DAG runs and results*: Allows users to track the execution status and outcomes of tasks within the DAGs.
- **Metastore**
 - *Keep system state*: Acts as a database to maintain the current state of the system.
 - *Track executed DAG nodes*: Records information about which parts of the DAG have been executed.
- **Scheduler**
 - *Parse DAGs*: Reads and interprets the DAG files to understand the workflow structure.
 - *Track completed dependencies*: Ensures that tasks are executed in the correct order by monitoring dependencies.
 - *Add tasks to execution queue*: Schedules tasks by placing them in a queue for execution.
 - *Schedule tasks*: Determines when tasks should be executed based on their dependencies and schedules.
- **Queue**
 - *Tasks ready for execution*: Holds tasks that are prepared and waiting to be executed.
 - *Tasks picked up by Workers*: Workers retrieve tasks from this queue to execute them.
- **Workers**
 - *Pick up tasks from Queue*: Retrieve tasks from the queue for execution.

- *Execute tasks*: Perform the actual work defined by the tasks.
- *Register task outcome in Metastore*: Record the results of task execution back into the Metastore for tracking and monitoring.

9 / 13: Airflow: Concepts

Airflow: Concepts

- Each DAG run represents a data interval
 - E.g., a DAG scheduled `@daily`
 - Data interval starts at midnight, ends at midnight next day
- DAG scheduled after data interval ends
- Logical date
 - Simulate scheduler running DAG/task for a specific date
 - Even if physically run now

9 / 13

- **Airflow: Concepts**
- **Each DAG run represents a data interval**
 - When we talk about a *DAG* (Directed Acyclic Graph) in Airflow, we're referring to a collection of tasks that are organized to run in a specific order. Each time this DAG is executed, it covers a specific period, known as a *data interval*.
 - For example, if a DAG is scheduled to run `@daily`, it means that each run of the DAG will cover a 24-hour period. This interval starts at midnight and ends at midnight the next day. This helps in organizing and processing data that is generated or collected daily.
- **DAG scheduled after data interval ends**
 - Airflow schedules the execution of a DAG after the data interval it covers has ended. This means that if a DAG is set to run daily, it will be scheduled to run after the full day has passed. This ensures that all data for that interval is available for processing.
- **Logical date**
 - The *logical date* is a concept in Airflow that allows you to simulate running a DAG or a task for a specific date, even if you are executing it at a different time. This is useful for backfilling or rerunning tasks for past dates. It helps maintain consistency in data

processing by ensuring that tasks are executed as if they were run on the intended date, regardless of when they are actually executed.

10 / 13: Airflow: Tutorial

Airflow: Tutorial

- Follow Airflow Tutorial in README
- From the tutorial for Airflow

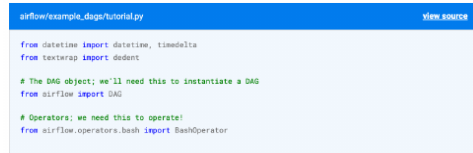
10 / 13

- **Airflow: Tutorial**
 - **Follow Airflow Tutorial in README**
 - * Airflow is a platform used to programmatically author, schedule, and monitor workflows. It's a powerful tool for managing complex data pipelines.
 - * The README file typically contains essential information and instructions for getting started with Airflow. It serves as a guide to help you set up and understand the basics of how Airflow works.
 - * By following the tutorial in the README, you'll learn how to install Airflow, configure it, and create your first workflow or Directed Acyclic Graph (DAG).
 - * This step is crucial for beginners as it provides hands-on experience and helps in understanding the core concepts of Airflow, such as tasks, operators, and scheduling.
 - **From the tutorial for Airflow**
 - The tutorial is designed to give you a practical understanding of Airflow's capabilities and how to use them effectively.
 - It usually includes examples of creating and managing workflows, which are essential for automating and orchestrating tasks in data engineering and data science projects.
 - By completing the tutorial, you will gain insights into how Airflow can be integrated into your data infrastructure to improve efficiency and reliability.
 - This foundational knowledge is important for anyone looking to leverage Airflow for managing data workflows in a scalable and maintainable way.

11 / 13: Airflow: Tutorial

Airflow: Tutorial

- Script describes DAG structure as Python code
 - No computation inside DAG code
 - Defines DAG structure and metadata (e.g., scheduling)
- **Scheduler** executes code to build DAG
- **BashOperator** creates task wrapping Bash command



```

airflow/example_dags/tutorial.py
view source

from datetime import datetime, timedelta
from coverage import Coverage

# The DAG object; we'll need this to instantiate a DAG
from airflow import DAG

# Operators; we need this to operate!
from airflow.operators.bash import BashOperator

```

11 / 13

- **Script describes DAG structure as Python code**
 - In Airflow, a Directed Acyclic Graph (DAG) is defined using Python scripts. This script outlines the structure and flow of tasks without performing any actual computations within the DAG code itself.
 - The script is responsible for defining the DAG's structure and metadata, such as scheduling information, which dictates when and how often the DAG should run.
- **@Scheduler@ executes code to build DAG**
 - The Airflow Scheduler is a component that reads the DAG definitions and executes the tasks according to the specified schedule. It ensures that tasks are run in the correct order and at the right time.
- **BashOperator creates task wrapping Bash command**
 - The **BashOperator** is used to create tasks that execute Bash commands. This operator allows you to run shell commands as part of your workflow, making it versatile for various automation tasks.

The code snippet shows the necessary imports for creating a DAG and using the **BashOperator**. It includes importing essential modules like **datetime** for scheduling and **DAG** from Airflow to define the workflow.

12 / 13: Airflow: Tutorial

Airflow: Tutorial

- Dict with various default params to pass to the DAG constructor
 - E.g., different set-ups for dev vs prod
- Instantiate the DAG

```

airflow/example_dags/tutorial.py
# These args will get passed on to each operator
# You can override them on a per-task basis during operator initialization
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'email': ['airflow@example.com'],
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    # 'queue': 'bash_queue',
    # 'pool': 'backfill',
    # 'priority_weight': 10,
    # 'end_date': datetime(2016, 1, 1),
    # 'wait_for_downstream': False,
    # 'dag': dag,
    # 'sla': timedelta(hours=2),
    # 'execution_timeout': timedelta(seconds=300),
    # 'on_failure_callback': some_function,
    # 'on_success_callback': some_other_function,
    # 'on_retry_callback': another_function,
    # 'sla_miss_callback': yet_another_function,
    # 'trigger_rule': 'all_success'
}

with DAG(
    'tutorial',
    default_args=default_args,
    description='A simple tutorial DAG',
    schedule_interval=timedelta(days=1),
    start_date=datetime(2021, 1, 1),
    catchup=False,
    tags=['example'],
) as dag:

```

12 / 13

- **Dict with various default params to pass to the DAG constructor**
 - This refers to a dictionary in Python that holds default parameters for a Directed Acyclic Graph (DAG) in Apache Airflow.
 - These parameters can include settings like the owner of the DAG, email notifications, retry policies, and more.
 - *Example:* You might have different configurations for development and production environments, such as different email addresses or retry settings.
- **Instantiate the DAG**
 - This involves creating a new DAG instance using the parameters defined in the dictionary.
 - The code snippet shows how to set up a simple tutorial DAG with a description, schedule interval, start date, and tags.
 - *Key Point:* The `catchup` parameter is set to `False`, meaning past DAG runs that were missed will not be executed when the DAG is started.

13 / 13: Airflow: Tutorial

Airflow: Tutorial

- DAG defines tasks by instantiating **Operator** objects
 - Default params passed to all tasks
 - Can be overridden explicitly
- Use a Jinja template
- Add tasks to the DAG with dependencies

```
airflow/example_dags/tutorial.py
```

```
t1 = BashOperator(
    task_id='print_date',
    bash_command='date',
)
```

```
t2 = BashOperator(
    task_id='sleep',
    depends_on_past=False,
    bash_command='sleep 5',
    retries=3,
)
```

```
airflow/example_dags/tutorial.py
```

```
templated_command = dedent(
    """
    {% for i in range(5) %}
    echo '{{ ds }}'
    echo '{{ macros.ds_add(ds, 7) }}'
    echo '{{ params.my_param }}'
    {% endfor %}
    """
)

t3 = BashOperator(
    task_id='templated',
    depends_on_past=False,
    bash_command=templated_command,
    params={'my_param': 'Parameter 1 passed in'},
)
```

```
t1 >> [t2, t3]
```

13 / 13

- **DAG defines tasks by instantiating Operator objects**
 - In Airflow, a Directed Acyclic Graph (DAG) is a collection of tasks organized to reflect their relationships and dependencies. Each task is created using an **Operator**, which is a predefined template for a specific type of work.
 - **Default params passed to all tasks:** You can set default parameters for tasks in a DAG, which helps maintain consistency and reduces redundancy.
 - **Can be overridden explicitly:** While default parameters are useful, you can override them for specific tasks if needed, providing flexibility.
- **Use a Jinja template**
 - Jinja templates allow dynamic generation of task commands. This is useful for creating commands that change based on variables or other conditions.
- **Add tasks to the DAG with dependencies**
 - Tasks are added to the DAG with specified dependencies, ensuring they execute in the correct order. The code snippet shows **t1** executing before **t2** and **t3**.

The code examples illustrate how to define tasks using **BashOperator**, set dependencies, and use Jinja templates for dynamic command generation. This setup is crucial for automating complex workflows efficiently.